Introduction & Setup

Week 1 · Foundations module · Textbook Chapter 1

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

Friday, August 21, 2026

This week at a glance

Welcome to Web Data Science. The semester starts Thursday, so this week we meet **Friday only** — today is orientation and your first contribution to the course textbook.

Friday | Welcome, the book & your first revision

- What web data science is — and why it matters
- How the class works and how you're graded
- The textbook: what it is, how it was made, how it's built
- Git & GitHub — and **you file a real issue today**

The **Monday / Wednesday / Friday** rhythm begins next week.

Companion notebook

`ch-01-introduction.ipynb`

Before Monday

Work through `handouts/setup.md`: Anaconda, Git, and a **free** GitHub account. Bring a laptop — today included.

1. Welcome

Meet your instructor

Brian C. Keegan — Associate Professor of Information Science.

- Grew up outside Las Vegas, Nevada
- MIT: Mechanical Engineering and Science, Technology & Society
- A year as a bartender and oral historian
- Ph.D. at Northwestern (Media, Technology & Society); postdoc in computational social science at Northeastern
- Data scientist at Harvard Business School
- CU Boulder Information Science, 2016–present

`brianckeegan.com`

What I study

- How Wikipedia covers breaking news
- Public-interest data science
- Migration across platforms (Threads, Mastodon, Bluesky)
- Online extremism and moderation

My students and I rely on web data every day — but researchers' access to it is rapidly disappearing.

What is web data science?

Web data science is the practice of *retrieving* data from the web, *parsing* it into structured formats, and *analyzing* it to produce knowledge.

It differs from general data science in one crucial way: your first challenge is not modeling or visualization — it is **getting the data in the first place**.

The web is the largest, most dynamic, and most contested data source ever created. It can tell us how misinformation spreads, how housing costs shift, and how online communities govern themselves.

Not your usual dataset

The web is **not a database**. Data arrives as:

- HTML tables buried in tracking scripts
- JSON from rate-limited APIs
- scanned PDFs of council minutes
- posts that might vanish tomorrow

Textbook Ch. 1, "Why Web Data Science?"

Data in the public interest

Web data science grew out of **computational social science**, **data journalism**, and **civic technology** — traditions that share one idea: data fluency is a form of power that should serve the public.

- **ProPublica, "Machine Bias" (2016)** — scraped court records exposed racial bias in criminal risk-assessment algorithms.
- **The Markup, "Citizen Browser"** — a recruited panel revealed how Facebook's feed distributed political content.

The tools are not neutral

The same skills that make public information **accessible** can also enable surveillance. Who benefits? Who is harmed? These questions run through every chapter.

Textbook Ch. 1, "Social History & Public Interest." The field's founding vision: Lazer et al. (2009).

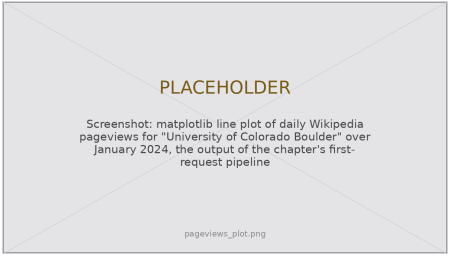
The loop that structures everything

Every chapter in this course follows the same arc:

retrieve → parse → analyze

- **Retrieve** a page or API response with `requests`
- **Parse** it into records — HTML, JSON, XML, or PDF
- Load into a **pandas** DataFrame
- **Analyze** or visualize the result

The specifics change; the arc does not. Recognizing it early gives you a mental model for every new chapter.



A first pipeline: CU Boulder's daily Wikipedia pageviews.

You'll run this yourself in `handouts/setup.md` before Monday. Textbook Ch. 1, "Your First Request."

The data science mindset

Technical fluency is not enough. Four dispositions will serve you all semester:

- **Growth mindset** — ability develops through effort. Treat each error as a learning signal, not evidence of inadequacy.
- **Computational thinking** — decompose, recognize patterns, abstract, and specify unambiguous steps (Wing 2006).
- **The hacker ethic** — sharing, openness, curiosity, and a bias toward action. The libraries you'll use exist because people shared their work.
- **Scientific norms** — communalism, skepticism, and reproducibility. Document your methods; question your data; report limitations.

This is normal

You will meet libraries you've never used and pages that resist your parsing. Frustration is the job, not a detour from it.

Textbook Ch. 1, "The Data Science Mindset."

2. How this class works

The weekly rhythm

Starting next week, every week runs on the same three-beat rhythm:

Monday | Concept + notebook intro

- Read the chapter; I introduce the week's ideas and the companion notebook

Wednesday | Notebook lab

- Hands-on — we work and debug the chapter's code together, live

Friday | Textbook revisions

- Propose and peer-review pull requests that improve the book

Concept, then practice, then contribution.

This week is the exception — Friday only. The full rhythm begins next week.

How you're evaluated

Your grade is **what you build**, not what you memorize:

- **Notebook Labs — 30%**
  weekly, hands-on data-collection work
- **Textbook Revisions — 30%**
  pull requests + peer reviews that improve the book
- **Final Project — 40%**
  a research design + real data collection + analysis

No exams

No midterm, no final. You demonstrate learning by **doing the work of a web data scientist** — collecting data, contributing to a shared resource, and building a project.

Percentages are the course total; weekly labs are not individually high-stakes.

Documentation is the job

Your previous classes may have discouraged online resources. **The training wheels are off.** Finding, reading, and applying documentation is *the* core professional skill — not a shortcut, not cheating.

Bookmark the docs you'll live in: `requests`, `BeautifulSoup`, `pandas`, `matplotlib`.

"It's not working" is not a question. Say what you **tried**, what you **expected**, what **happened**, and what the **error message** says.

Credit your sources

Used a blog post, a Q&A answer, docs, or an AI tool to solve something beyond class?

Just include a link in your code. Undocumented advanced tricks are a reliable signal of uncredited help.

Textbook Ch. 1, "Documentation as Professional Practice."

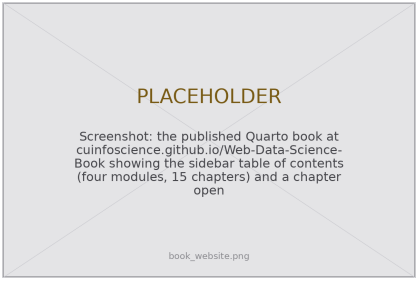
3. The textbook

A book built for this course

There is no book to buy. Our text is open source, and it is shaped to the course rather than the other way around:

- **15 chapters, 15 teaching weeks** — roughly one chapter per week
- **Four modules** matching ours: Foundations, Documents, APIs, Practice
- Every chapter ships a **companion notebook**, plus exercises and a "Common Issues to Debug" list
- Code is **not pre-executed** (`eval: false`) — you run it, so you meet the errors yourself

Read it at `cuinfoscience.github.io/Web-Data-Science-Book`



The published book — free, open, and ours to improve.

The syllabus schedule and this book's table of contents are the same document, seen twice.

How this book was written

I'll be direct about it: **portions of this book were drafted with large language models**, then reviewed, edited, and verified by me. The repository documents the process, and an appendix discloses it.

That process is fast and fluent — and it produces three predictable weaknesses:

- **Confident text that is subtly wrong** — a selector that never matched, a parameter that doesn't exist
- **Explanations from the far side of understanding** — correct, but skipping where you actually get stuck
- **Examples that decay** — the web changes underneath the book

Why this matters to you

You are meeting this material **for the first time**. That makes you the best possible reviewer for exactly the failures a fluent draft hides.

Your revisions aren't busywork. They're the verification step.

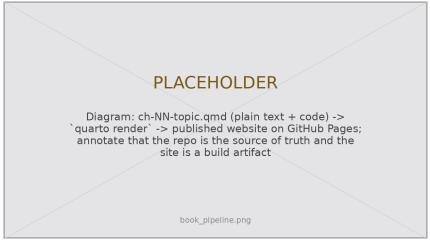
Same standard as the syllabus asks of you: use the tools, verify the output, and disclose the assistance.

How it's built and published

The book is a **Quarto** project living in a **GitHub** repository. Nothing about it is a mystery box:

1. Each chapter is a plain-text `.qmd` file — Markdown with executable code blocks
2. `quarto render` turns the `.qmd` files into the website
3. GitHub publishes the result at `cuinfoscience.github.io`
4. `references.bib` holds the citations; `claude.md` documents editorial voice and chapter structure

So "propose a revision" concretely means: **edit a `.qmd` file** and ask for it to be merged.



Source → render → published site.

The repository is the source of truth. The website is a build artifact of it.

4. Git & GitHub

Git & GitHub

Git records the history of a project — who changed what, when, and why — and lets many people work on it without overwriting each other.

GitHub hosts Git projects online and adds the social layer: proposing, discussing, and reviewing changes.

Create a free account at `github.com` with your `colorado.edu` email to unlock student features. Our textbook lives at:

`github.com/cuinfoscience/Web-Data-Science-Book`

The book is a living, open repository.

Setup instructions are in `handouts/setup.md` — have this working before Monday.

Four objects you'll use all semester

- **Repository** — the project: every file and its full history.
- **Issue** — a written report: "here is a problem." Costs nothing but a browser.
- **Pull request** — a proposed change: "here is the fix, side by side with the original."
- **Review** — a response to a PR: comment line-by-line, then approve or request changes.

Issue vs. pull request

An **issue** says *something is wrong*. A **pull request** says *here is the correction* — and shows the exact diff.

Issues are cheap and fast; PRs are the real contribution. **Today you'll file an issue.** From Week 2 on, you'll open pull requests.

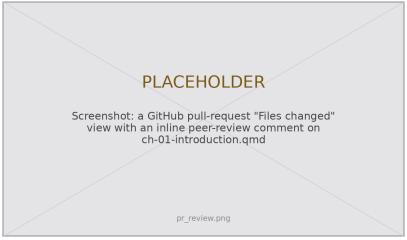
This is how essentially all open-source software gets built — including every library in this course.

Your Friday workflow

From Week 2 onward, every Friday repeats these four steps:

1. **Fork / branch** the book repository
2. **Edit** a chapter's `.qmd` source; commit with a clear message
3. **Open a pull request** describing *what* you changed and *why*
4. **Review** two classmates' PRs — kind, specific, and actionable

A pull request is a **proposal**, not an edit. Your change sits next to the original so peers and I can comment line-by-line before anything merges.



Contributions & reviews are the Textbook Revisions grade (30%). Full workflow walkthrough in Week 2.

5. Activity · Propose a revision

Today's activity: file your first issue

Goal: by the end of class, every one of you has filed one real GitHub issue proposing a change to Chapter 1.

- You need only a **browser** and a **GitHub account** — no Git, no fork, no local copy
- Work in pairs; file individually
- Read **Chapter 1** on the published site — you've just heard the lecture version, so you know what it should have told you

This is a genuine contribution to a public resource, not a practice exercise.

No account yet?

Make one now (2 minutes, `colorado.edu` email), or pair with someone who has one and draft together.

Timebox

5 read · **5** pick & diagnose · **8** write & file · **5** share out

Where did the chapter fail you?

That question — not "what could be added in principle" — starts every revision. Sort what you noticed into one of three families:

Broken

It is **wrong**, or no longer works.

Show: what you ran, and what happened instead.

Unclear

It is right, but you **couldn't follow it**.

Show: where you got stuck, and why.

Missing

You **needed something** that wasn't there.

Show: the gap is real, not merely possible.

Your inexperience is the qualification

Nobody who already knows this material can tell me where a first-time reader falls off. You can — but only while you're still a first-time reader. That window closes in about a week.

Seven kinds of revision

#	Type	You noticed…	Your contribution includes	Size
1	Code drift	An example errors or returns nothing	The error, the fix, what changed upstream	M
2	Stale figure	A screenshot doesn't match today's UI	A fresh screenshot, same filename & crop	S
3	Common issue	An error not in "Common Issues to Debug"	Symptom → cause → fix, in the list's format	S
4	Missing figure	You sketched it yourself to understand it	The figure, a caption, and alt text	M
5	Reading	A claim with no source behind it	The citation in `references.bib` + why it belongs	S
6	Exercise	An exercise is ambiguous or unverifiable	Expected output, a rubric, or a starter cell	M
7	Explanation	You reread a passage three times	The rewrite + what confused you the first time	M

Type 3 is the easiest high-value contribution in the book — it converts your lost debugging time into somebody else's saved time.

Size = review effort. **S**: a few lines. **M**: a paragraph or code block. Bigger than M? File an issue first.

The full framework, with worked examples of each type: `handouts/revision-framework.md`

How issues & pull requests are graded

Weekly, out of **10 points** — the Textbook Revisions grade (30%).

Criterion	Full marks	Pts
Located	Chapter, section & file named	2
Evidenced	I can reproduce what you hit	3
Actionable	A concrete change, scoped to one thing	3
Reviews	Two peer reviews, specific, with a verdict	2
	Total	**10**

Most weeks, most people land at **7–9**. A 10 means someone could act on your proposal without asking a single follow-up question.

Merged is not the bar

You're graded on the **proposal and the review**, not on whether I merge it. A well-evidenced PR I decline for editorial reasons earns full credit. A merged one-character typo fix does not.

Little or no credit

- Bulk typo PRs
- "This is confusing" with no location or proposal
- Duplicates — check open issues first, then *review* theirs
- Undisclosed AI text you didn't verify

Writing it: the skeleton

Every issue and every PR description uses the same five fields:

- **Title** — `<type>: <specific thing> in Ch. N`
- **Location** — chapter, section, and the `.qmd` file
- **Problem** — what you did, expected, and got. Paste the code and output.
- **Why** — who this affects, in one sentence
- **Proposal** — the concrete change you'd make

Can't fill in **Location** and **Problem** with specifics? You don't have a revision yet — you have a hunch. Go back and reproduce it.

Worked example

Title: Common issue: kernel/environment mismatch in Ch. 1

Location: `ch-01-introduction.qmd`, "Setting Up Your Environment"

Problem: Installed `requests` in `webdata`, but the notebook raised `ModuleNotFoundError`. The kernel was pointing at base.

Why: Likely to hit anyone using conda environments — cost me 20 minutes.

Proposal: Add to "Common Issues": check `sys.executable` in the notebook.

One change per issue or PR. Three fixes in one PR is three reviews wearing one coat.

Your turn

1. **Read** Chapter 1 on the published site — skim for the parts you'd have needed today (5 min)
2. **Pick one** problem. Name its family and its type from the table (5 min)
3. **Check** the repo's open issues — if yours exists, comment on it instead (1 min)
4. **Write and file** it using the five-field skeleton (8 min)
5. **Share out** — a few of you read yours aloud (5 min)

`Issues` → `New issue` — that's the whole mechanism.

Next week

Your issue becomes your **first pull request** — you'll fix the thing you found.

Stuck choosing? "The chapter never told me X, and I needed X" is always a legitimate issue.

6. Before you go

Before Monday

Work through `handouts/setup.md` — about 45 minutes:

1. **Python, via Anaconda** — plus a `webdata` environment and the core libraries
2. **Git** — installed and configured with your name and email
3. **A free GitHub account** — with your `colorado.edu` address

The handout ends with a two-cell check: retrieve a page, then pull Wikipedia pageviews into a DataFrame. **If it runs, you're ready.**

Don't suffer in silence

Email me **before Monday** if setup fails — arriving with a broken environment costs you the lab.

Can't access a laptop or install software? Contact me **immediately** so we can arrange an accommodation.

Also read Textbook Ch. 2 (Ethics, law & responsible collection) before Monday.

The web is not a database.
It's a chaotic, evolving ecosystem of documents, APIs, and norms.

Your first challenge isn't modeling —
it's getting the data at all.

Key takeaways

1. Web data science is **retrieve → parse → analyze —** and the hard part is getting the data at all.
2. Every week has a rhythm: **Monday** concept, **Wednesday** notebook lab, **Friday** textbook-revision pull requests.
3. The textbook is **open, Quarto-built, and AI-drafted under human review —** which is exactly why your first-time-reader perspective is valuable.
4. Revisions sort into **Broken / Unclear / Missing** and seven concrete types. Graded on **location, evidence, actionability, and reviews —** not on whether I merge them.
5. Your grade is what you build: Labs 30%, Revisions 30%, Final Project 40% **— no exams.**

Next week: **Ethics, law & responsible collection** — `robots.txt`, Terms of Service, and how to collect data without doing harm.

Reading: Textbook Ch. 1–2. Related: Salganik (2018); Wilson et al. (2017).